

Prueba de Caja Blanca

Gestion de inventario “Los Huertos del Juank”

Integrantes:

Esteban Cuichan, Johan Catota, Eatan Ayala, Katerine Caizaluisa

Fecha: 2026/06/22

Requisito funcional N°: 4

Que de MTZ de HU: (REQ004 – Registro de Clientes). “Registrar ficha personal de clientes con código único, para identificar a los compradores y generar reportes”

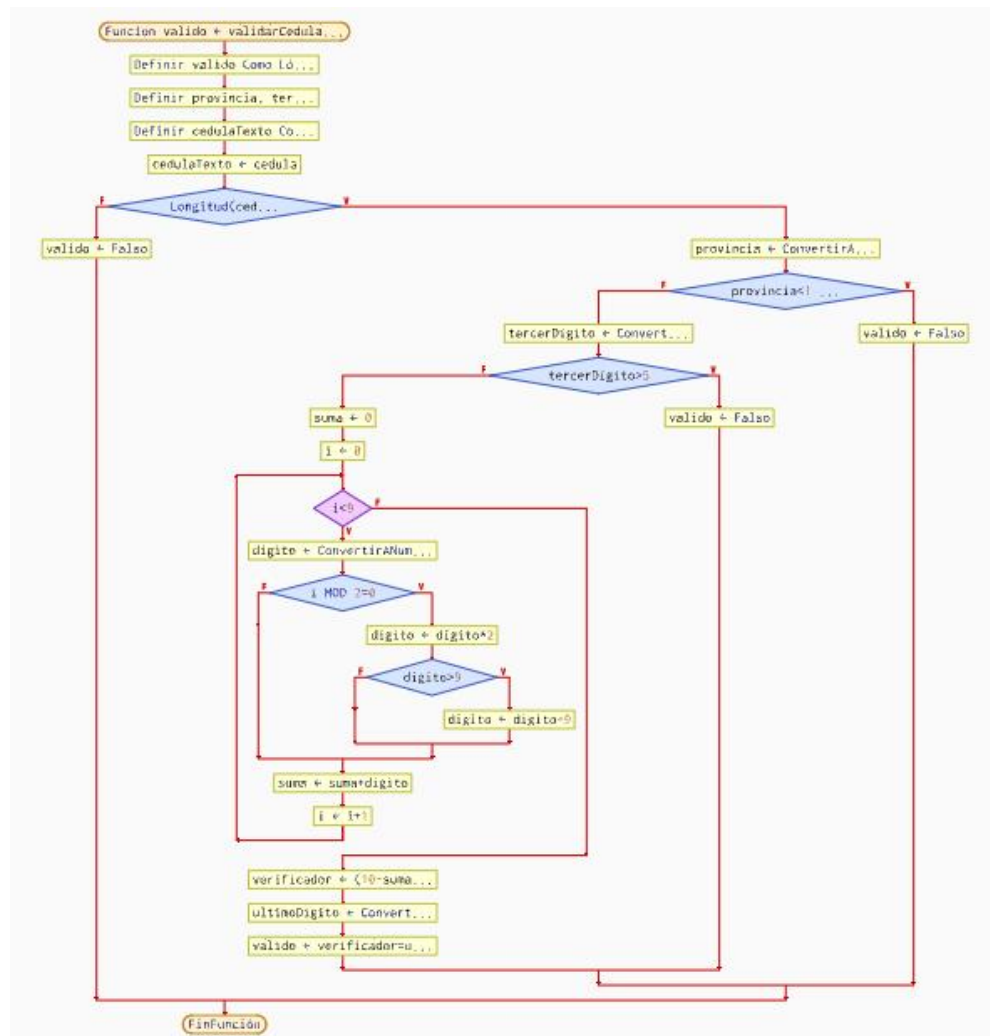
CÓDIGO FUENTE PARA EL REGISTRO DE CLIENTES (VALIDACIÓN DE CÉDULA)

Función: validarCedula(String cedula), ubicada en controllers/ClienteService.java. Esta función se ejecuta antes de guardar o modificar un cliente, dentro del método validarDatos().

```
private boolean validarCedula(String cedula) {  
    if (!cedula.matches("\\d{10}")) return false;  
    int provincia = Integer.parseInt(cedula.substring(0, 2));  
    if (provincia < 1 || provincia > 24) return false;  
    int tercerDigito = Character.getNumericValue(cedula.charAt(2));  
    if (tercerDigito > 5) return false;  
    int suma = 0;  
    for (int i = 0; i < 9; i++) {  
        int digito = Character.getNumericValue(cedula.charAt(i));  
        if (i % 2 == 0) {  
            digito *= 2;  
            if (digito > 9) digito -= 9;  
        }  
        suma += digito;  
    }  
    int verificador = (10 - (suma % 10)) % 10;  
    return verificador == Character.getNumericValue(cedula.charAt(9));  
}
```

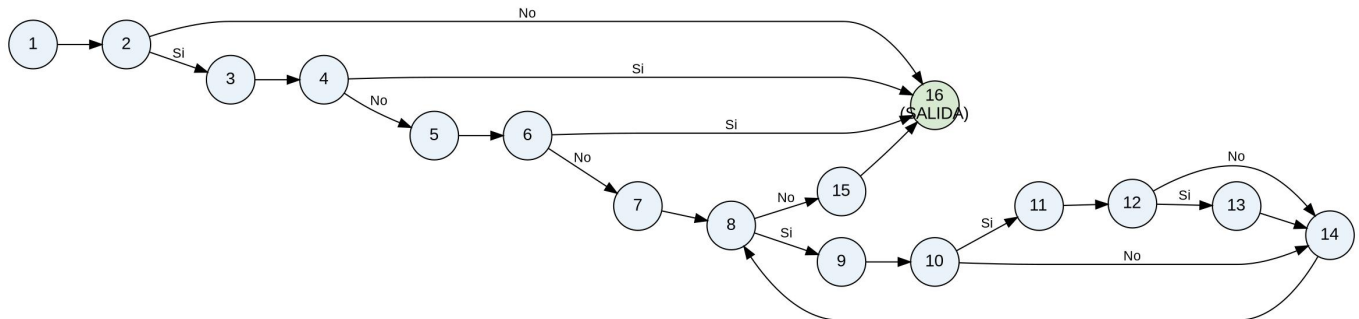
DIAGRAMA DE FLUJO (DF) – validarCedula()

Representación del flujo de control de la función del numeral 1, identificando cada instrucción y cada punto de decisión del método validarCedula().



GRAFO DE FLUJO (GF) – validarCedula()

Grafo de flujo construido en base al DF del numeral anterior. Cada nodo numérico corresponde a un bloque de código o decisión; las tres instrucciones “return false” (líneas 2, 4 y 6 del DF) convergen en el nodo de salida 16, junto con el “return” final.



IDENTIFICACIÓN DE LAS RUTAS (Camino básico) – validarCedula()

Determinadas en base al GF del numeral anterior. Se identifican 7 rutas independientes (igual al valor de la complejidad ciclomática calculada en el siguiente numeral):

R1: 1-2-16 → Cédula con formato inválido (no son 10 dígitos numéricos). Retorna false de inmediato.

R2: 1-2-3-4-16 → Formato válido, pero el código de provincia está fuera del rango 01-24. Retorna false.

R3: 1-2-3-4-5-6-16 → Formato y provincia válidos, pero el tercer dígito es mayor a 5. Retorna false.

R4: 1-2-3-4-5-6-7-8-(9-10-11-12-13-14-8)*-15-16 → Todos los datos válidos; en el bucle, alguna posición par ($i\%2==0$) produce $\text{digito} * 2 > 9$, por lo que se ejecuta la resta -9.

R5: 1-2-3-4-5-6-7-8-(9-10-11-12-14-8)*-15-16 → Todos los datos válidos; en el bucle, alguna posición par produce $\text{digito} * 2 \leq 9$, por lo que NO se ejecuta la resta -9.

R6: 1-2-3-4-5-6-7-8-(9-10-14-8)*-15-16 → Todos los datos válidos; en el bucle, alguna posición es impar ($i\%2!=0$), por lo que el dígito no se multiplica.

R7: 1-2-3-4-5-6-7-8-...-8-15-16 → Cédula completamente válida: el bucle termina ($i=9$) y el dígito verificador calculado coincide con el último dígito de la cédula → return true.

COMPLEJIDAD CICLOMÁTICA – validarCedula()

Se calcula de las siguientes formas, en base al GF del numeral anterior (N = 16 nodos, A = 21 aristas, P = 6 nodos predicado: líneas 2, 4, 6, 8 -bucle for-, 10 e 12):

Fórmula 1 (nodos predicado):

$$V(G) = \text{Nodos predicado (IF-THEN-ELSE / bucles)} + 1$$

$$V(G) = 6 + 1 = 7$$

Fórmula 2 (aristas y nodos):

$$V(G) = A - N + 2$$

$$V(G) = 21 - 16 + 2 = 7$$

DONDE:

P: Número de nodos predicado (decisiones: 3 condicionales “if” de validación + 1 condición del bucle for + 2 condicionales internas al bucle = 6)

A: Número de aristas del grafo de flujo (21)

N: Número de nodos del grafo de flujo (16)

Conclusión: la complejidad ciclomática de validarCedula() es $V(G) = 7$, lo que indica que se requieren 7 casos de prueba (rutas R1 a R7) para lograr una cobertura de caminos básicos del método.